O padrão **Singleton** é um padrão de projeto criacional que garante que uma classe tenha **apenas uma única instância** em toda a aplicação e fornece um **ponto de acesso global** a ela. [1, 2]

É muito utilizado em cenários como conexões de banco de dados, gerenciadores de configuração ou *pools* de threads. [1]

---

Estrutura básica

Toda implementação do Singleton em Java baseia-se em três pilares: [1]

1. **Construtor privado**: Impede que classes externas usem o `new` para instanciar o objeto.
2. **Atributo estático privado**: Guarda a única instância da classe.
3. **Método estático público** (geralmente chamado `getInstance()`): Retorna a instância única. [1]

Exemplo (Singleton com inicialização preguiçosa / ***Lazy Initialization***):

java

```
public class Singleton {
    // 1. Atributo estático privado que armazena a única instância
    private static Singleton instancia;

    // 2. Construtor privado para evitar o uso de "new Singleton()"
    private Singleton() {
    }

    // 3. Método estático público para acessar a instância
    public static Singleton getInstancia() {
        if (instancia == null) {
            instancia = new Singleton();
        }
        return instancia;
    }
}
```

Use o código com cuidado.

---

Principais formas de implementação

- **Lazy Initialization (Preguiçosa):** A instância só é criada quando o método `getInstancia()` for chamado pela primeira vez (como no exemplo acima). [1]

- **Eager Initialization (Ansiosa):** A instância é criada assim que a classe é carregada pela JVM: [1]

  java

  ```
  public class Singleton {
      private static final Singleton instancia = new Singleton();
      private static Singleton() {}
      public static Singleton getInstancia() {
          return instancia;
      }
  }
  ```

  Use o código com cuidado.
- **Thread-Safe com Double-Checked Locking:** Usado em ambientes multithread para evitar que duas threads criem instâncias simultâneas sem perder desempenho com sincronização excessiva: [1, 2]

  java

  ```
  public class Singleton {
      private static volatile Singleton instancia;
      private Singleton() {}

      public static Singleton getInstancia() {
          if (instancia == null) {
              synchronized (Singleton.class) {
                  if (instancia == null) {
                      instancia = new Singleton();
                  }
              }
          }
          return instancia;
      }
  }
  ```

  Use o código com cuidado.
- **Enum Singleton (Recomendado por Joshua Bloch):** Considerada a forma mais segura em Java, pois a própria JVM impede a criação de múltiplas instâncias e protege contra ataques de reflexão e serialização:

  java

  ```
  public enum SingletonEnum {
      INSTANCIA;
  ```

```
        public void algumMetodo() {
            // lógica aqui
        }
    }
```

Use o código com cuidado.

*(Acessado via `SingletonEnum.INSTANCIA.algumMetodo()`)*. [1]

---

Cuidados e desvantagens

- **Dificuldade em testes unitários:** Como mantém estado global e acoplamento, pode dificultar a criação de testes isolados.
- **Problemas com concorrência:** Se não for *thread-safe*, múltiplas threads podem criar instâncias separadas ao mesmo tempo.
- **Violação de princípios:** Pode ferir o princípio da responsabilidade única (SRP) e agir como uma variável global disfarçada. [1, 2]

Mais detalhes e tutoriais podem ser consultados no Refactoring.Guru ou no guia da DevMedia. [1, 2]